

A PROJECT REPORT ON
CHURNSHIELD AI – CUSTOMER CHURN PREDICTION
SYSTEM

An End-to-End Machine Learning and Flask-Based Web Application for Customer Retention Intelligence

Submitted in partial fulfillment of the requirements for the award of

Bachelor of Computer Applications (BCA)

in Department of Computer Applications & Information Technology

SUBMITTED BY:

Soumyadip Porel

Roll No: [BCA/2023-2026/042]

Reg No: [REG/BCA/7891024]

UNDER THE GUIDANCE OF:

[Prof. / Dr. Internal Guide Name]

Assistant Professor, Department of Computer Applications

Department of Computer Applications & Information Technology

[NAME OF COLLEGE / ACADEMIC INSTITUTE]

Academic Year: 2025 – 2026

EXECUTIVE PROJECT SUMMARY

This report presents ChurnShield AI – Customer Churn Prediction System, an end-to-end Machine Learning and Flask-based web application developed by Soumyadip Porel for final-year BCA project requirements. The system predicts customer churn probability and classifies account holders into four distinct risk levels (Low, Medium, High, Very High) to empower retail banking retention.

Key Model Performance Indicators:

- Final Model: HistGradientBoostingClassifier
- Decision Threshold: 0.35 (Optimized for proactive churn recall)
- Overall Accuracy: 85.50%
- Precision: 64.89%
- Recall: 62.65%
- F1 Score: 63.75%
- ROC-AUC: 86.76%

Confusion Matrix Breakdown (N = 2,000 holdout test records):

- True Negatives (TN): 1455
- False Positives (FP): 138
- False Negatives (FN): 152
- True Positives (TP): 255

Top Permutation Importance Features:

1. Age
2. Products Number
3. Country Germany
4. Active Member
5. Balance

Application Architecture:

- Backend: Python, Flask 3.x, Scikit-learn, Joblib, Gunicorn
- Frontend: HTML5, CSS3, JavaScript, Responsive UI, Dynamic SVG Radial Gauge
- Dataset: Bank Customer Churn Prediction.csv (10,000 records, 12 raw columns, 11 final encoded features, 0 missing, 0 duplicates)
- Workflow: 11-step ML pipeline from data ingestion to cloud deployment.

CHAPTER 1 – INTRODUCTION

1.1 Background

In contemporary retail banking and financial institutions, customer retention has emerged as one of the pivotal determinants of sustainable profitability and long-term institutional stability. Financial service providers operate in an increasingly saturated and competitive market where customer acquisition costs (CAC) typically exceed customer retention costs by a factor of five to seven times. The phenomenon of customer attrition—frequently referred to as "customer churn"—occurs when an existing account holder closes their accounts, shifts their primary transactions to an alternate institution, or permanently terminates their business relationship with the bank.

Historically, financial institutions relied on retrospective reporting and post-attrition exit interviews. Such reactive approaches, however, identify churn only after the customer has already terminated their relationship, eliminating any viable opportunity for preventive customer relationship management (CRM) interventions. With the emergence of statistical computing and computational intelligence, predictive machine learning (ML) models offer institutions the analytical capability to identify behavioral precursors and underlying risk patterns before defection occurs.

The project titled "ChurnShield AI – Customer Churn Prediction System" addresses this critical challenge by providing an end-to-end, automated machine learning pipeline coupled with a production-grade web application framework. By systematically evaluating banking transaction indicators and customer profiles, ChurnShield AI estimates individual churn probabilities, classifies customers into distinct risk tiers, and presents actionable insights to banking branch managers and relationship officers.

1.2 Problem Statement

Retail banks encounter substantial financial attrition each fiscal quarter due to unexpected customer churn. The fundamental challenges within conventional banking operations include:

1. **Inability to Detect Latent Risk:** Traditional rule-based business heuristics fail to detect non-linear interactions among multidimensional variables such as customer age, product holding count, international branch geography, and balance fluctuations.
2. **Data Imbalance:** In standard banking cohorts, customers who churn comprise a minority class (typically between 15% and 25%), while the vast majority remain retained. Generic classification models evaluated on standard accuracy frequently suffer from high false-negative rates, failing to identify genuine churners.
3. **Suboptimal Decision Thresholds:** Standard machine learning classifiers default to a rigid decision threshold of 0.50. In financial risk prevention, the penalty of failing to flag a churning customer (False Negative) is substantially costlier than dispatching a retention promotion to a loyal customer (False Positive).
4. **Lack of Operational Deployment:** Many machine learning implementations remain confined to analytical Jupyter notebooks, lacking user-friendly, responsive interfaces suitable for frontline bank branch staff and credit officers.

1.3 Motivation

The motivation behind developing ChurnShield AI stems from bridging the operational gap between theoretical machine learning methodologies and practical software engineering deployment in modern banking. While complex ensemble algorithms achieve strong theoretical benchmark performance in research literature, frontline banking staff require accessible, interpretable, and responsive web interfaces.

Furthermore, as a final-year Bachelor of Computer Applications (BCA) capstone project, this initiative synthesizes foundational software development concepts—data structures, Python programming, statistical data preparation, model optimization, web development, REST API design, and system testing—into a single, cohesive, production-grade deliverable that follows rigorous software engineering standards.

1.4 Objectives

The primary objectives of the ChurnShield AI project are formally enumerated as follows:

1. To conduct comprehensive Exploratory Data Analysis (EDA) on a structured bank customer dataset of 10,000 records to identify the core drivers of attrition.
2. To preprocess, clean, encode, and engineer feature vectors suitable for robust machine learning model training without data leakage.
3. To benchmark candidate classification algorithms—specifically Logistic Regression, Random Forest, and HistGradientBoostingClassifier—using standard evaluation metrics.
4. To calibrate and optimize the classification decision threshold from the standard 0.50 down to 0.35, thereby optimizing recall for churn detection.
5. To serialize the finalized high-performing model and deploy it via an asynchronous Flask web framework with RESTful JSON endpoints.
6. To construct an intuitive, accessible frontend user interface featuring customer intake forms, dynamic probability gauges, multi-tier risk classification (Low, Medium, High, Very High), and session prediction history tracking.

1.5 Scope and Delimitations

The project scope encompasses the complete lifecycle from offline dataset ingestion to client-side risk dashboard presentation. Specifically, the system processes 11 key demographic, financial, and product features to compute an empirical probability score between 0.00 and 1.00.

Delimitations of the project must be explicitly recognized: Predictions generated by ChurnShield AI represent statistical probability estimates based on historical patterns in the training cohort and do not constitute an absolute guarantee of customer retention or defection. Furthermore, the system processes single-customer queries and session-based client tracking, operating within a dedicated Flask and Gunicorn execution environment.

CHAPTER 2 – LITERATURE AND TECHNOLOGY OVERVIEW

2.1 Customer Churn Prediction in Banking

Customer churn prediction represents a classic binary classification problem in supervised machine learning. In the banking sector, customer relationships are contract-based (e.g., credit card facilities, loans, savings accounts). Early churn prediction systems relied upon statistical logistic regression and univariate scorecards. Modern literature demonstrates that gradient boosted decision trees and tree-ensemble paradigms substantially outperform linear counterparts due to their capability to discover intricate feature cross-products and non-linear threshold splits without requiring complex manual transformations.

2.2 Supervised Machine Learning Principles

Supervised machine learning operates on labeled training pairs (X_i, y_i) , where X_i in $\mathbb{R}^d$ denotes the d -dimensional input feature vector representing customer attributes, and y_i in $\{0, 1\}$ represents the binary ground truth label (0 for retained, 1 for churned). The objective function minimizes empirical risk over a defined loss function, balancing model bias against variance to maximize generalization on unseen validation splits.

2.3 Classification Algorithms Under Study

During the analytical phase of this project, three representative classification paradigms were evaluated:

1. Logistic Regression: A generalized linear model utilizing the sigmoid link function. It provides high interpretability and serves as the baseline benchmark, but struggles to model high-order interaction effects present in banking datasets.
2. Random Forest: An ensemble bagging technique constructing an array of de-correlated decision trees trained on bootstrap samples with random feature subsets. While robust against overfitting, training and inference latency can scale linearly with tree count.
3. HistGradientBoostingClassifier: An advanced, histogram-based gradient boosted decision tree algorithm inspired by LightGBM and integrated natively within Scikit-learn. It discretizes continuous numerical features into integer-valued bins (typically 256 bins), dramatically reducing split-finding computational complexity from $O(n_{\text{samples}} * n_{\text{features}})$ to $O(n_{\text{bins}} * n_{\text{features}})$. It natively handles non-linear relationships, features monotonic constraints, and scales exceptionally well on tabular datasets.

2.4 HistGradientBoosting Architecture

The HistGradientBoostingClassifier constructs trees in a sequential stage-wise additive manner. At each boosting iteration m , a new weak decision tree $h_m(x)$ is fitted to the negative gradient (pseudo-residuals) of the binomial log-likelihood loss function evaluated at the current ensemble approximation $F_{\{m-1\}}(x)$. Continuous feature binning eliminates the need for expensive sorting at each node split, resulting in memory efficiency, rapid inference, and superior ROC-AUC metrics on tabular data.

2.5 Web Framework and Deployment Technologies

The deployment architecture utilizes Python Flask—a lightweight, WSGI-compliant micro-framework chosen for its minimal overhead, deterministic routing, and seamless interoperability with Python scientific libraries (Scikit-learn, NumPy, Pandas). Static assets and dynamic user interaction are orchestrated using HTML5, modern CSS3 styling, and modular JavaScript to handle asynchronous fetch API transactions, client-side input validation, dynamic risk gauge rendering, and DOM manipulation without full-page reloads. Production serving is powered by Gunicorn (Green Unicorn), a battle-tested WSGI HTTP server executing multi-worker pre-fork concurrency.

CHAPTER 3 – SYSTEM ANALYSIS

3.1 Analysis of the Existing System

In many conventional banking workflows, retention efforts operate reactively rather than proactively. Key limitations of existing institutional systems include:

- **Post-Facto Identification:** Intervention triggers occur only when account balances drop to zero, cards are deactivated, or formal account closure requests are lodged at physical branches.
- **Fragmented Heuristics:** Customer relationship officers manually inspect rudimentary indicators (e.g., credit score alone) without statistical weighting of multi-factor relationships such as geographic branch location, product density, or age group.
- **Lack of Automated Probability Scoring:** Branch managers lack standardized numerical risk percentages and uniform risk tier classifications.
- **High Latency and Cost:** Manual portfolio reviews consume excessive staff hours and fail to scale across portfolios of tens of thousands of active clients.

3.2 Proposed System Architecture

The proposed ChurnShield AI system introduces an automated, real-time prediction framework that addresses all weaknesses identified in the existing system:

- **Instant Predictive Intelligence:** Computes precise churn probabilities in less than 50 milliseconds upon submitting customer profile attributes.
- **Multi-Tier Risk Categorization:** Maps the continuous probability output into four distinct, business-interpretable risk tiers: Low Risk ($< 25\%$), Medium Risk ($25\% - 50\%$), High Risk ($50\% - 75\%$), and Very High Risk ($> 75\%$).
- **Decision Threshold Optimization:** Applies a 0.35 calibrated decision threshold specifically tuned for early warning detection, ensuring maximum recall without excessive false alarms.
- **Transparent Interface:** Provides branch officers with dynamic visual gauges, interactive session history logging, prediction counters, and instant re-analysis capabilities.

3.3 Software and Hardware Requirements

Hardware Requirements for Development and Inference:

- **Processor:** Dual-Core 64-bit x86/ARM processor (Intel Core i3 / AMD Ryzen 3 or higher).
- **Memory (RAM):** Minimum 4 GB RAM (8 GB recommended for model training and grid search).
- **Storage:** 500 MB of available secondary storage for repository, serialized models, and dataset.
- **Display:** Standard 1080p display resolution with responsive mobile viewport support.

Software and Library Stack Requirements:

- **Operating System:** Linux (Ubuntu 22.04 LTS), macOS, or Microsoft Windows 10/11.

- Runtime Environment: Python 3.10+ runtime.
- Core Scientific Libraries: Pandas (data manipulation), NumPy (vectorized array operations), Scikit-learn (HistGradientBoosting, preprocessing, metrics), Joblib (model serialization).
- Data Visualization Libraries: Matplotlib, Seaborn (exploratory data charts and confusion matrix plots).
- Backend Web Framework: Flask 3.x, Gunicorn WSGI server.
- Frontend Technologies: HTML5 Semantic Markup, CSS3 Flexbox/Grid, Modern JavaScript (ES6+).

3.4 Functional Requirements

The functional capabilities specified for ChurnShield AI comprise:

FR-1 (Customer Data Intake): The web interface must present an intuitive, validated input form capturing credit score, age, tenure, balance, products number, credit card possession, active membership status, estimated salary, country (Germany/Spain/France), and gender.

FR-2 (Client-Side Input Validation): Numeric fields must enforce boundary constraints (e.g., credit score 300–850, age 18–100, non-negative balances) before submission.

FR-3 (RESTful Inference Endpoint): The Flask server must expose a POST /predict API endpoint accepting JSON payloads and returning probability scores and risk labels.

FR-4 (Model Feature Alignment): The backend must execute exact one-hot feature encoding alignment corresponding to the trained model schema.

FR-5 (Probability & Risk Tier Computation): The system must output exact probability percentages alongside corresponding risk classification badges (Low, Medium, High, Very High).

FR-6 (Session History Tracking): The interface must maintain a dynamic session table recording all predictions conducted during the browser session, complete with a total prediction counter and a "Clear History" action.

FR-7 (Reset & Re-Analysis): An "Analyze Another Customer" control must clear input fields and reset the form state for subsequent queries.

3.5 Non-Functional Requirements

NFR-1 (Latency & Performance): The end-to-end inference round-trip time between form submission and visualization render must execute in under 100 milliseconds.

NFR-2 (Reliability & Robustness): The API must gracefully intercept malformed requests, missing attributes, or invalid data types, returning HTTP 400 status codes with informative JSON error messages.

NFR-3 (Usability & Responsiveness): The user interface must adapt cleanly across desktop, tablet, and mobile displays without horizontal overflow or misaligned form inputs.

NFR-4 (Maintainability & Modularity): ML training scripts (train.py), web server routes (app.py), serialized artifacts (/model), and assets (/static, /templates) must maintain strict directory separation.

CHAPTER 4 – DATASET AND DATA PREPROCESSING

4.1 Dataset Description

The empirical foundation of the ChurnShield AI project utilizes the benchmark banking dataset file entitled "Bank Customer Churn Prediction.csv". The dataset comprises exactly 10,000 unique customer records and 12 initial feature columns characterizing account holder demographic profiles, bank relationship duration, financial account balances, product usage, and historical defection status.

The target attribute is "churn", represented as a binary indicator where value 0 denotes an active, retained customer, and value 1 denotes a churned customer who severed their banking relationship.

4.2 Feature Schema and Descriptions

The dataset incorporates demographic, financial, and behavioral attributes described in Table 4.1 below. Irrelevant identifier fields such as customer IDs or row indices were omitted to prevent spurious correlation.

4.3 Data Cleaning and Quality Verification

Rigorous diagnostic audits were conducted during initial data ingestion:

1. Missing Value Analysis: Exhaustive scanning across all 10,000 rows revealed exactly 0 missing, null, or NaN entries across all 12 columns. Consequently, synthetic imputation or sample deletion was unnecessary.
2. Duplicate Record Inspection: A duplicate query across customer records verified 0 duplicate rows, confirming that every record represents an autonomous customer instance.
3. Range and Outlier Sanity Checks: Feature distributions were bounded within physically valid financial domains (e.g., credit scores strictly bounded between 350 and 850, age values between 18 and 92, and non-negative balances).

4.4 Categorical Encoding and Final Feature Vectors

Machine learning algorithms require numerical input matrices. Categorical variables in the raw dataset were transformed through deterministic one-hot encoding:

- Geography / Country: The original categorical feature contained three classes: France, Germany, and Spain. To avoid multicollinearity (the dummy variable trap), "France" was treated as the baseline reference category, generating two binary columns: "country_Germany" and "country_Spain".
- Gender: The binary feature was transformed into "gender_Male" (1 for Male, 0 for Female).

This transformation yielded the finalized 11-feature input vector utilized throughout training and real-time production inference: credit_score, age, tenure, balance, products_number, credit_card, active_member, estimated_salary, country_Germany, country_Spain, and gender_Male.

4.5 Target Class Distribution and Imbalance

A fundamental characteristic of the dataset is its natural class imbalance. Out of 10,000 total customers:

- Class 0 (Retained / Non-churned): 7,963 customers (79.63%)
- Class 1 (Churned / Positive): 2,037 customers (20.37%)

This ~4:1 imbalance ratio has profound implications for machine learning design: A naive baseline model predicting

"retained" for every customer would achieve an apparent accuracy of 79.63% while completely failing to identify a single churning customer. This reality necessitates threshold calibration and comprehensive evaluation via Precision, Recall, F1 Score, and ROC-AUC rather than raw classification accuracy alone.

CHAPTER 5 – EXPLORATORY DATA ANALYSIS (EDA)

5.1 Exploratory Objectives and Methodology

Exploratory Data Analysis was conducted using Python, Pandas, Matplotlib, and Seaborn to inspect distributions, examine inter-feature correlations, and uncover behavioral patterns distinguishing churned customers from retained ones.

5.2 Key Empirical Findings and Observations

Analysis of bivariate and multivariate distributions uncovered five key findings:

1. **Age Impact:** Age exhibited the highest statistical correlation with churn. Customers in the 45–60 age bracket demonstrated a churn rate exceeding 50%, whereas customers aged 25–35 exhibited churn rates below 10%. Older clientele frequently face wealth transition phases or seek specialized institutional wealth services.
2. **Product Number Discrepancy:** Customers utilizing exactly two bank products exhibited the highest loyalty and lowest churn rate (~8%). In sharp contrast, customers holding 3 or 4 products displayed alarmingly high churn rates exceeding 80%, indicating potential product complexity dissatisfaction, high maintenance fees, or administrative friction.
3. **Geographic Disparities:** While customers in France and Spain showed churn rates around 16%, customers in Germany exhibited churn rates of approximately 32%—double the rate of the other regions. This highlights significant regional market dynamics.
4. **Membership Activity Status:** Inactive members ("active_member" = 0) demonstrated churn rates almost double those of active banking members, confirming that account inactivity serves as a primary behavioral indicator of imminent defection.
5. **Balance Dynamics:** Retained customers possessed both low/zero balances and high balances, but customers with balances between €100,000 and €150,000 in specific demographic tiers exhibited elevated churn risks, signifying high-value flight risk.

5.3 Analytical Visualizations

Visual distributions generated during the exploratory phase confirmed distinct multi-modal distributions in customer age and product count. Figure 5.1 illustrates the pronounced shift in churn probability across age cohorts, and Figure 5.2 demonstrates the relationship between geography, product portfolio depth, and attrition probability.

CHAPTER 6 – MACHINE LEARNING METHODOLOGY

6.1 End-to-End ML Pipeline Architecture

The modeling architecture followed a structured, 11-step machine learning workflow:

1. Dataset inspection & schema verification
2. Data preparation & identifier filtering
3. Exploratory Data Analysis & visual correlation discovery
4. Train/Test split partition (80:20 stratified split)
5. Baseline model training across candidate algorithms
6. Hyperparameter optimization and tuning
7. Probability calibration & threshold sensitivity analysis
8. Final model evaluation on unseen test partition
9. Model serialization via Joblib
10. Flask REST API integration
11. Full-stack web application deployment

6.2 Train / Test Split Protocol

The preprocessed dataset of 10,000 records was partitioned using an 80:20 train-to-test split ratio with stratification based on the target churn variable. This guaranteed that both the training set (8,000 samples) and the holdout test set (2,000 samples) retained identical class proportions (approximately 20.37% churned, 79.63% retained). The test split remained completely quarantined until final model validation.

6.3 Candidate Model Selection and Training

Three distinct algorithms representing diverse inductive biases were trained on the 8,000-sample training partition:

- Model 1: Logistic Regression (L2 regularization, solver="lbfgs")
- Model 2: Random Forest Classifier (n_estimators=100, max_depth=12)
- Model 3: HistGradientBoostingClassifier (max_iter=150, learning_rate=0.08, min_samples_leaf=20)

HistGradientBoostingClassifier demonstrated clear superiority across ROC-AUC and generalization capacity on tabular continuous/categorical mixed inputs.

6.4 Hyperparameter Optimization

Systematic tuning of HistGradientBoostingClassifier was performed using cross-validated search over learning rate, maximum iterations, maximum leaf nodes, and L2 regularization to balance learning capacity against overfitting risk. Table 6.1 delineates the finalized parameter configuration.

6.5 Decision Threshold Analysis (Selection of 0.35)

In standard binary classification, an observation is assigned to the positive class if the predicted probability $P(y=1|X) \geq 0.50$. However, in banking customer churn prediction, the business costs of classification errors are highly asymmetric:

- False Positive (FP): A loyal customer is flagged as a potential churner. The bank incurs a minimal marketing retention cost (e.g., an automated courtesy email, a personalized promotional interest rate offer, or a loyalty voucher).
- False Negative (FN): A customer intending to churn is erroneously predicted as retained. No retention outreach is triggered, and the customer defects, forfeiting their lifetime transaction fees, deposits, and future relationship value.

Therefore, a threshold sensitivity analysis was conducted across thresholds ranging from 0.20 to 0.60. At the standard 0.50 threshold, model recall was below 48%, missing over half of all churners. By lowering the decision threshold to 0.35, the model captured 62.65% of all churning customers (Recall) while maintaining a robust Precision of 64.89% and an overall Accuracy of 85.50%. This balanced operating point achieved an optimal F1 Score of 63.75%.

6.6 Formal Evaluation Metric Formulations

To ensure rigorous scientific assessment, standard evaluation metrics derived from the confusion matrix were implemented. Each metric is formally defined with its mathematical formulation below.

CHAPTER 7 – SYSTEM DESIGN AND ARCHITECTURE

7.1 Overall Architectural Overview

ChurnShield AI is architected as a modular, three-tier client-server application adhering to strict separation of concerns: the Presentation Layer (Frontend), the Application Service Layer (Flask REST API), and the Machine Learning Persistence Engine (Serialized Model & Preprocessing Pipeline).

7.2 Machine Learning Pipeline Module

The ML pipeline component encapsulates data parsing, validation, vectorized transformation, and probability inference. Once trained via `train.py`, the model object and associated configuration metadata are serialized into the `model/` directory using Joblib for rapid, zero-overhead memory loading at server startup.

7.3 Flask Backend Controller Architecture

The application backend is implemented in `app.py` utilizing the Flask micro-framework. It hosts two primary route handlers:

1. Route GET `"/`: Serves the primary single-page application dashboard rendered from `templates/index.html`.
2. Route POST `"/predict"`: Accepts an asynchronous JSON payload containing customer profile attributes, executes schema validation, formats input into an 11-column NumPy array, triggers `model.predict_proba()`, applies the 0.35 decision threshold, maps the score to the appropriate risk tier, and returns a structured JSON response.

7.4 Frontend UI and Visualization Subsystem

The client interface provides a modern, responsive web portal created with HTML5, CSS3, and JavaScript. It features real-time input validation, a dynamic SVG radial probability gauge, high-contrast risk badges, an interactive session history ledger, and an interactive prediction counter.

7.5 End-to-End Prediction Flow Sequence

The operational execution sequence follows a clean deterministic lifecycle:

- Step 1: The user enters customer attributes into the web form and clicks "Predict Churn Risk".
- Step 2: Client JavaScript intercepts the submission, validates value boundaries, and renders an active loading animation.
- Step 3: An asynchronous `fetch()` POST request transmits the JSON payload to the `/predict` API endpoint.
- Step 4: Flask parses the payload, validates feature types, and constructs the 11-element feature vector.
- Step 5: The serialized `HistGradientBoostingClassifier` computes class probabilities via `predict_proba()`.
- Step 6: The backend evaluates the probability against threshold 0.35 and assigns one of four risk levels (Low, Medium, High, Very High).
- Step 7: Flask returns the JSON response containing probability, binary prediction, risk category, and timestamp.
- Step 8: Client JavaScript updates the dynamic risk visualization, increments the prediction counter, appends a record to the session history table, and renders the "Analyze Another Customer" option.

7.6 Repository and Project Directory Structure

The repository follows industry-standard conventions separating data artifacts, serialized models, research notebooks, backend controllers, and client templates.

CHAPTER 8 – IMPLEMENTATION DETAILS

8.1 Dataset Processing and Scripting

Data preprocessing logic is codified within `train.py`. The script ingests "Bank Customer Churn Prediction.csv" from the `dataset/` folder using Pandas. Feature columns are isolated, dummy encoding is applied to categorical dimensions with `drop_first=False` for precise column naming, and the resulting DataFrame is cast to 32-bit floating point NumPy arrays to optimize memory utilization.

8.2 Model Training Pipeline

The training execution instantiates Scikit-learn's `HistGradientBoostingClassifier` configured with the calibrated hyperparameter suite. Cross-validation verifies consistent validation loss, and the full training split (8,000 samples) is fitted.

8.3 Model Serialization via Joblib

To decouple model training from web serving, the trained model artifact along with feature names and decision threshold settings are serialized into the `model/` directory using Joblib compression.

8.4 Flask Prediction API Implementation

The web server in `app.py` initializes Flask, loads the serialized model into memory once during process startup, and establishes the POST `/predict` endpoint.

8.5 Frontend Client Implementation

The frontend utilizes responsive HTML5 and CSS3 styling with modular JavaScript. Form submissions trigger client-side validation, dispatch asynchronous fetch API calls with JSON encoding, activate CSS-based loading spinners, and dynamically update the DOM with risk levels, probability indicators, and session history rows.

CHAPTER 9 – RESULTS AND PERFORMANCE EVALUATION

9.1 Comparative Model Performance Analysis

All three candidate models were evaluated on the identical holdout test partition containing 2,000 samples. Table 9.1 documents the comparative performance metrics across algorithms.

9.2 Final Model Performance Metrics

The selected HistGradientBoostingClassifier with calibrated threshold 0.35 achieved the following performance metrics on the independent test dataset (N = 2,000):

- Overall Accuracy: 85.50%
- Precision (Positive Predictive Value): 64.89%
- Recall (Sensitivity / True Positive Rate): 62.65%
- F1 Score (Harmonic Mean): 63.75%
- ROC-AUC Score: 86.76%

These results validate that the model successfully balances high discrimination capacity (ROC-AUC 86.76%) with practical recall, detecting nearly two out of every three churners while preserving high precision.

9.3 Confusion Matrix Analysis

The confusion matrix on the 2,000-sample test set (comprising 1,593 retained customers and 407 churned customers) demonstrates the classification breakdown under decision threshold 0.35:

- True Negatives (TN): 1,455 instances (Correctly identified retained customers)
- False Positives (FP): 138 instances (Retained customers classified as churn risks)
- False Negatives (FN): 152 instances (Churned customers missed by the model)
- True Positives (TP): 255 instances (Correctly detected churning customers)

The confusion matrix demonstrates that out of 407 actual churners in the holdout split, the model successfully identified 255 (62.65% Recall). Furthermore, of the 393 total accounts flagged as potential churn risks (TP + FP = 255 + 138), 255 were actual churners, yielding a solid 64.89% precision.

9.4 Permutation Feature Importance Analysis

To ensure model transparency and explainability, permutation feature importance was computed across the test partition. In permutation importance, the values of each feature are randomly shuffled while keeping all other features constant, and the resulting decrease in model score (ROC-AUC) is measured. The top five most influential features are:

1. age: Shuffling age produced the steepest decline in model discrimination, reinforcing that life-stage transitions represent the strongest single predictor of account defection.

2. products_number: Disrupting product holding counts severely compromised predictive power due to the extreme non-linear churn risk associated with holding 3 or 4 bank products.
3. country_Germany: Shuffling the German regional indicator caused significant performance drops, reflecting the substantial market-specific churn differential in that geography.
4. active_member: Account inactivity status ranked fourth, serving as a direct behavioral precursor to attrition.
5. balance: Account balance magnitude ranked fifth, capturing financial capacity and flight risk among high-balance account holders.

9.5 Sample Prediction Case Evaluations

To illustrate the practical operation of ChurnShield AI, Table 9.3 provides representative test cases across diverse customer demographic and financial profiles.

CHAPTER 10 – WEB APPLICATION INTERFACE

10.1 Application Dashboard and Architecture

The ChurnShield AI web application is designed as an interactive single-page dashboard tailored for banking credit analysts and customer relationship officers. The interface integrates the following core capabilities:

- Clean, intuitive customer intake form with organized financial and demographic sections.
- Real-time input boundary validation with informative feedback cues.
- Asynchronous inference submission with loading animation indicators.
- Dynamic probability visualization using animated SVG radial progress meters.
- Multi-tier risk categorization badges (Low, Medium, High, Very High).
- Session-based prediction history tracking ledger with a live prediction counter.
- "Analyze Another Customer" reset functionality and clear history options.

10.2 Customer Information Input Form

The customer information form captures all eleven predictive attributes grouped into three logical panels:

1. Demographic Details: Age (18–100), Gender (Male/Female), and Geography (France/Germany/Spain).
2. Banking Relationship: Tenure (0–10 years), Products Count (1–4), Credit Card Status (Yes/No), and Active Membership Status (Yes/No).
3. Financial Metrics: Credit Score (300–850), Account Balance (EUR), and Estimated Annual Salary (EUR).

HTML5 min/max attributes and JavaScript regular expressions ensure all submitted inputs conform strictly to valid financial and biological domains.

10.3 Prediction Result and Dynamic Risk Gauge

Upon processing the API response, the dashboard reveals a dedicated result card featuring an animated radial gauge displaying the exact churn probability percentage. The interface dynamically applies color-coded risk visual styling according to the established risk tiers described in Table 10.1.

10.4 Prediction History and Session Counter

To support branch officers conducting multi-client reviews, the frontend records each prediction in a dynamic session history table. The table displays customer age, geography, product count, predicted probability, assigned risk tier, and timestamp. A summary counter tracks total predictions evaluated during the active session, with a "Clear History" button allowing one-click ledger resets.

CHAPTER 11 – SYSTEM VERIFICATION AND TESTING

11.1 Testing Methodology and Objectives

System verification followed a structured testing regime encompassing unit testing, input validation testing, REST API integration testing, and responsive user interface validation across varied screen viewports.

11.2 Functional and Input Validation Testing

Validation testing verified that boundary edge cases, negative numerical values, and non-numeric inputs are intercepted before dispatching inference requests. Table 11.1 documents representative test cases and observed outcomes.

11.3 Flask REST API Endpoint Testing

Automated API integration tests were executed against the POST /predict endpoint using Python requests and curl scripts. The suite verified HTTP status code compliance, JSON response structure, execution latency, and error handling for malformed payloads.

11.4 User Interface and Cross-Browser Testing

The frontend interface was tested across Google Chrome, Mozilla Firefox, Apple Safari, and Microsoft Edge across desktop (1920x1080), laptop (1366x768), tablet (768x1024), and smartphone (375x667) screen viewports. All form controls, radial gauge animations, and history tables adjusted cleanly without visual clipping or horizontal overflow.

CHAPTER 12 – PROJECT LIMITATIONS

12.1 Academic and Technical Limitations

While ChurnShield AI successfully meets its stated engineering objectives, several inherent limitations must be formally acknowledged:

1. **Statistical Estimates vs. Deterministic Outcomes:** Machine learning predictions represent empirical probability estimates derived from historical patterns. They do not constitute an infallible guarantee that a given customer will or will not defect.
2. **Cross-Sectional Static Snapshot:** The underlying dataset represents a cross-sectional point-in-time snapshot. It does not incorporate fine-grained time-series telemetry such as monthly transaction frequency trends or clickstream interactions within mobile banking applications.
3. **Single-Institution Demographic Context:** The dataset reflects banking clientele across three European countries (France, Germany, Spain). Model weights may not generalize directly to developing economy financial institutions without local recalibration.
4. **In-Memory Session History:** The frontend prediction history resides within browser session memory and does not persist across browser restarts or synchronize to an external relational database.
5. **Omission of Unstructured Feedback:** The system currently analyzes structured tabular features and does not process qualitative unstructured customer sentiment from customer support logs or branch complaints.

CHAPTER 13 – FUTURE ENHANCEMENTS

13.1 Potential Future Developments

Future iterations of the ChurnShield AI platform could explore several enhancements:

1. **Batch CSV Scoring:** Extend the web application to support bulk CSV uploads, enabling branch managers to score entire customer portfolios containing tens of thousands of records simultaneously.
2. **Longitudinal Time-Series Analysis:** Incorporate temporal transaction velocity and recurring balance trends using Recurrent Neural Networks (RNN) or temporal transformer architectures.
3. **Explainable AI (XAI) Integration:** Incorporate localized SHAP (SHapley Additive exPlanations) force plots directly within the web UI to provide customer-specific attribution breakdowns for each individual prediction.
4. **Enterprise CRM & Database Integration:** Connect the Flask application to enterprise relational databases (e.g., PostgreSQL) with role-based access control (RBAC) and automated webhook notifications to CRM platforms like Salesforce.
5. **Prescriptive Intervention Engine:** Expand beyond predictive churn classification into prescriptive analytics by recommending personalized retention incentives (e.g., fee waivers or preferential interest rates) tailored to specific customer risk drivers.

CHAPTER 14 – CONCLUSION

14.1 Project Conclusion

The "ChurnShield AI – Customer Churn Prediction System" project successfully demonstrates the end-to-end conception, training, calibration, and deployment of a machine learning application designed to tackle customer attrition in retail banking.

Through systematic exploratory data analysis, feature engineering, and rigorous benchmarking across candidate classification algorithms, the HistGradientBoostingClassifier was established as the optimal predictive model. By strategically tuning the decision threshold to 0.35, the system achieved a well-calibrated operating point balancing 62.65% Recall, 64.89% Precision, 85.50% Accuracy, and an ROC-AUC score of 86.76% on a holdout test partition of 2,000 customers.

The serialized model was integrated into a responsive Flask web application featuring a customer information form, dynamic probability visualization, multi-tier risk classification (Low, Medium, High, Very High), session prediction tracking, input boundary validation, and a RESTful prediction API. This capstone project successfully demonstrates the translation of statistical machine learning into practical, usable software engineering deliverables.

14.2 Summary of Key Achievements

The key technical achievements accomplished throughout this project include:

- Clean data pipeline processing 10,000 banking records with zero data leakage.
- Rigorous benchmark evaluation establishing HistGradientBoostingClassifier as superior to Logistic Regression and Random Forest.
- Threshold calibration from 0.50 to 0.35, boosting churn recall to 62.65% while maintaining 64.89% precision.
- Computation of permutation feature importance confirming age, product number, German geography, active membership, and balance as primary churn drivers.
- Lightweight, low-latency Flask REST API providing sub-50ms inference response times.
- Professional, responsive client dashboard featuring animated probability gauges, risk tiers, and session history management.
- Comprehensive test suite validating input boundaries, API payloads, and cross-browser rendering.

REFERENCES

Academic and Technical References

- [1] Scikit-learn Documentation: "Histogram-Based Gradient Boosting," Scikit-learn Developers, 2024. [Online]. Available: <https://scikit-learn.org/stable/modules/ensemble.html#histogram-based-gradient-boosting>
- [2] Pedregosa, F., et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [3] Flask Documentation: "Flask: A Python Microframework," Pallets Projects, 2024. [Online]. Available: <https://flask.palletsprojects.com/>
- [4] McKinney, W., "Data Structures for Statistical Computing in Python," in Proceedings of the 9th Python in Science Conference, 2010, pp. 51–56.
- [5] Harris, C. R., et al., "Array programming with NumPy," Nature, vol. 585, pp. 357–362, 2020.
- [6] Hunter, J. D., "Matplotlib: A 2D Graphics Environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007.
- [7] Waskom, M., "Seaborn: statistical data visualization," Journal of Open Source Software, vol. 6, no. 60, p. 3021, 2021.
- [8] Fawcett, T., "An introduction to ROC analysis," Pattern Recognition Letters, vol. 27, no. 8, pp. 861–874, 2006.
- [9] Gunicorn Documentation: "Gunicorn WSGI HTTP Server for UNIX," 2024. [Online]. Available: <https://gunicorn.org/>
- [10] Breiman, L., "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.